

Commonly Asked ML Interview Questions

1. What is the difference between Parametric and Non Parametric Algorithms?
2. Difference between convex and non-convex cost function; what does it mean when a cost function is non-convex?
3. How do you decide when to go for deep learning for a project?
4. Give an example of when False positive is more crucial than false negative and vice versa
5. Why is "Naive" Bayes naive?
6. Give an example where the median is a better measure than the mean
7. What do you mean by the unreasonable effectiveness of data?
8. Why KNN is known as a lazy learning technique?
9. What do you mean by semi supervised learning?
10. What is an OOB error and how is it useful?
11. In what scenario decision tree should be preferred over random forest?
12. Why Logistic Regression is called regression?
13. What is Online Machine Learning? How is it different from Offline machine learning? List some of it's applications
14. What is No Free Lunch Theorem?
15. Imagine you are working with a laptop of 2GB RAM, how would you process a dataset of 10GB?
16. What are the main differences between Structured and Unstructured Data?
17. What are the main points of difference between Bagging and Boosting?
18. What are the assumptions of linear regression?
19. How do you measure the accuracy of a Clustering Algorithm?
20. What is Matrix Factorization and where is it used in Machine Learning?

Matrix factorization is a class of collaborative filtering algorithms used in recommender systems. Matrix factorization algorithms work by decomposing the user-item interaction matrix into the product of two lower dimensionality rectangular matrices.

21. What is an Imbalanced Dataset and how can one deal with this problem?

22. How do you measure the accuracy of a recommendation engine?

◆ 2. For Ranking or Top-N Recommendation Systems

(When the model suggests a list of items — e.g., “Top 10 movies you may like”)

We focus on **ranking metrics** that measure how relevant the recommended items are:

- **Precision@K** → Fraction of top-K recommended items that are actually relevant.
$$\text{Precision@K} = \frac{\text{\text{\# of relevant items in top K}}}{K}$$
- **Recall@K** → Fraction of all relevant items that appear in the top-K list.
$$\text{Recall@K} = \frac{\text{\text{\# of relevant items in top K}}}{\text{\text{Total relevant items}}}$$
- **F1-Score@K** → Harmonic mean of Precision and Recall.
- **Mean Average Precision (MAP)** → Averages precision across all recall levels.
- **Normalized Discounted Cumulative Gain (NDCG)** → Considers both relevance and ranking position (higher rank → higher gain).
- **Hit Rate (HR)** → Measures whether at least one relevant item appears in the top-K list.

23. What are some ways to make your model more robust to outliers?

To make a model robust to outliers, I first handle them at the data level using IQR or transformations, and scale features using median-based methods like RobustScaler. On the model side, I prefer robust loss functions like MAE or Huber Loss, or use tree-based models that naturally handle outliers well. Regularization also helps prevent overfitting to extreme values.

24. How can you measure the performance of a dimensionality reduction algorithm on your dataset?

“I evaluate dimensionality reduction based on the goal.

For PCA or autoencoders, I use reconstruction error or explained variance.

For methods like t-SNE or UMAP, I look at structure-preserving metrics such as trustworthiness and continuity.

And if it’s part of a downstream ML pipeline, I measure the task performance after reduction to ensure minimal information loss.”

25. What is Data Leakage? List some ways using which you can overcome this problem

Data leakage occurs when information that would not be available at prediction time is accidentally used during the model training process. This leads to overly optimistic results during training or validation, but poor generalization on unseen data. Essentially, the model “cheats” by learning from data it shouldn’t have access to. There are two main types of data leakage: **Target Leakage** and **Train-Test Contamination**. Target leakage happens when features include information derived from the target variable. For example, using a “loan repayment date” feature while predicting whether a person will default gives away future information. Train-test contamination occurs when information from the test or validation set leaks into the training set — for example, applying scaling, normalization, or feature selection on the entire dataset before splitting.

To prevent data leakage, several steps can be followed. First, always **split your data before preprocessing**. The train, validation, and test sets should be separated before applying any scaling, encoding, or feature engineering. Second, use **pipelines** (like scikit-learn’s `Pipeline` or `ColumnTransformer`) to ensure consistent, leakage-free preprocessing during cross-validation. Third, handle **time-series data carefully** by ensuring chronological splits — the model should only train on past data and predict the future. Fourth, when using **target encoding**, always use out-of-fold encoding or fit encoders only on training data. Fifth, perform **feature selection within each cross-validation fold**, not on the entire dataset, to avoid data snooping. Sixth, **track feature sources and audit their availability** — make sure all features exist at prediction time and are not derived from the target or future data. Finally, maintain a **true holdout test set** (or perform live A/B testing) to measure performance in a realistic environment.

In summary, **data leakage** is when a model unintentionally learns from future or external information, causing unrealistically high accuracy during training. To overcome this, I ensure proper data splitting before preprocessing, use pipelines, apply time-aware splits for temporal data, avoid target leakage, keep feature selection inside cross-validation, and always validate on a clean holdout set to ensure genuine performance.

26. What is Multicollinearity? How to detect it? List some techniques to overcome Multicollinearity.

Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other, meaning they convey similar information about the target variable. This makes it difficult for the model to determine the individual effect of each predictor, as changes in one variable can be explained by another. In such cases, regression coefficients become unstable — they can flip signs or change drastically with small variations in data — leading to unreliable interpretations, inflated standard errors, and poor model generalization.

To **detect multicollinearity**, several statistical methods can be used. The most common one is calculating the **Variance Inflation Factor (VIF)** for each predictor. A VIF value greater than 5 (sometimes 10) indicates a high degree of multicollinearity. Another approach is using the **correlation matrix** — if two features have a correlation coefficient above 0.8 or 0.9, they are likely collinear. Additionally, the **condition number** of the design matrix can be analyzed; a large condition number (greater than 30 or 100) suggests strong multicollinearity. Also, observing unstable or highly fluctuating regression coefficients after minor data changes is a practical signal of multicollinearity.

To **overcome multicollinearity**, several techniques can be applied. The simplest method is to **remove one of the correlated features** if both convey similar information. Another approach is to **combine correlated features** using techniques like averaging or **Principal Component Analysis (PCA)**, which creates uncorrelated components. Applying **regularization methods** such as **Ridge Regression (L2)** or **Lasso Regression (L1)** can also mitigate multicollinearity — Ridge shrinks correlated feature coefficients, while Lasso can eliminate redundant ones altogether. Additionally, **domain knowledge** can help retain the most meaningful predictors and remove those that are redundant or derived. Finally, increasing the **sample size** can sometimes reduce the adverse effects of multicollinearity by providing more data variability.

27. List some ways using which you can reduce overfitting in a model.

Overfitting occurs when a machine learning model learns the noise and random fluctuations in the training data instead of the true underlying patterns. As a result, it performs very well on the training set but fails to generalize to unseen data. Reducing overfitting involves techniques that improve the model's ability to generalize while preventing it from memorizing the training data.

Several effective methods can be used to reduce overfitting:

Train with More Data: The most straightforward approach is to collect more high-quality training data. A larger dataset helps the model capture the true data distribution instead of memorizing specific examples.

Simplify the Model: Use a less complex model that has fewer parameters or constraints. For instance, use a shallower decision tree or a smaller neural network architecture to reduce variance.

Regularization Techniques: Apply penalties on large coefficients to prevent the model from fitting noise.

- a. **L1 Regularization (Lasso):** Encourages sparsity by driving some coefficients to zero, effectively performing feature selection.
- b. **L2 Regularization (Ridge):** Penalizes large weights and keeps all coefficients small and stable.
- c. **Elastic Net:** Combines both L1 and L2 penalties for balanced control.

Cross-Validation: Use techniques like **k-fold cross-validation** to evaluate how the model performs on multiple splits of the data. This helps ensure the model generalizes well across different subsets of data.

Early Stopping: In iterative models like neural networks or gradient boosting, monitor validation loss during training and stop when it starts increasing — this prevents the model from learning noise.

Dropout (for Neural Networks): Randomly deactivate a fraction of neurons during each training iteration. This forces the network to learn more robust and redundant representations, improving generalization.

Data Augmentation: For image, audio, or text data, artificially expand the dataset by introducing transformations like rotations, flips, cropping, or adding noise. This exposes the model to more diverse variations of input data.

Pruning: In decision trees or ensemble methods, remove branches or weak learners that do not significantly contribute to performance. This reduces model complexity.

Ensemble Methods: Techniques like **bagging (Random Forests)** and **boosting (XGBoost, LightGBM)** combine multiple weak models to reduce variance and improve generalization.

Feature Selection: Remove irrelevant or redundant features that might introduce noise. Keeping only meaningful predictors helps the model focus on significant relationships.

Noise Injection: Slightly perturbing input data or labels during training can prevent the model from fitting too precisely to the training samples.

Parameter Tuning: Use grid search or random search to find optimal hyperparameters that balance model bias and variance.

28. What are the different types of bias in Machine Learning?

29. How do you approach a categorical feature with high cardinality?

30. Explain Pruning in Decision Trees and how it is done

31. What is ROC-AUC curve? List some of its benefits.

The ROC-AUC curve plots True Positive Rate against False Positive Rate for all classification thresholds, and the AUC quantifies the overall ability of a model to distinguish between classes. Its key benefits include being **threshold-independent, robust to class imbalance, providing a clear measure of discrimination, and allowing easy comparison between models**, making it one of the most popular metrics for evaluating binary classifiers.

32. What are kernels in SVM? Can you list some popular SVM kernels.

Kernels in SVM are functions that map input data into higher-dimensional space to make it linearly separable, enabling SVMs to handle non-linear problems. Popular kernels include **Linear, Polynomial, RBF (Gaussian)**, and **Sigmoid**. The choice of kernel and its parameters depends on the data's characteristics and the type of decision boundary needed. Kernels allow SVMs to efficiently model complex relationships using the "kernel trick" without explicitly computing high-dimensional mappings.

33. What is the difference between Gini Impurity and Entropy? Which one is better and why?

Gini Impurity and Entropy are metrics for measuring node impurity in decision trees. Gini measures the probability of misclassification, while Entropy measures the expected information required to classify a sample. Gini is computationally faster, whereas Entropy is slightly more sensitive to changes in class distribution. Both generally produce similar trees, but Gini is often preferred in practice for efficiency, while Entropy provides a more information-theoretic perspective.

34. Why does L2 regularization give sparse coefficients?

35. List some ways using which you can improve a model's performance.

36. Can PCA be used to reduce the dimensionality of a highly nonlinear dataset?

Principal Component Analysis (PCA) is a **linear dimensionality reduction technique**. It works by finding directions (principal components) in the feature space that **maximize variance** and are **linear combinations of the original features**.

❶ **Limitation for Nonlinear Data**

- PCA assumes that the data lies on or near a **linear subspace**.
 - If the dataset has **highly nonlinear structure** (e.g., curved manifolds, clusters in nonlinear arrangements), PCA cannot capture the underlying relationships well.
 - Applying PCA to nonlinear data may lead to **loss of important information** because it only considers linear correlations.
-

❷ **Alternatives for Nonlinear Dimensionality Reduction**

For highly nonlinear datasets, one can use **nonlinear dimensionality reduction techniques** such as:

Kernel PCA – Applies the kernel trick to map data to a higher-dimensional space and then performs PCA, capturing nonlinear patterns.

t-SNE (t-distributed Stochastic Neighbor Embedding) – Preserves local structure for visualization in 2D or 3D.

UMAP (Uniform Manifold Approximation and Projection) – Preserves both local and some global structure efficiently.

Isomap – Uses geodesic distances along the manifold to maintain nonlinear relationships.

Autoencoders (Neural Networks) – Learn nonlinear mappings from high-dimensional space to lower-dimensional latent space.

37. What's the difference between probability and likelihood?

Probability calculates the chance of observing data **given known parameters**, while likelihood measures how probable certain parameter values are **given observed data**. In short, probability is **data given parameters**, and likelihood is **parameters given data**. Likelihood is central to techniques like **Maximum Likelihood Estimation** in machine learning and statistics.

38. What cross-validation technique would you use on a time series data set. And why?

For time series datasets, I would use **Time Series Cross-Validation (Rolling Forecast or Walk-Forward Validation)** instead of traditional k-fold.

It ensures that training always happens on **past data** and validation on **future data**, preserving temporal order and preventing data leakage.

This method best mimics how the model will behave in real-world forecasting tasks.

39. Once a dataset's dimensionality has been reduced, is it possible to reverse the operation? If so, how? If not, why?

Yes, dimensionality reduction can be **reversed only in some cases**.

For **linear methods like PCA**, if all components are retained, the operation is fully reversible; if only a subset is kept, reconstruction is approximate due to information loss.

For **nonlinear methods like t-SNE or UMAP**, the process is **not reversible**, as they distort the data's geometry to preserve certain relationships and lose explicit mapping to the original feature space.

40. Why do we always need the intercept term in a regression model?

41. When Your Dataset Is Suffering From High Variance, How Would You Handle It?

When a dataset suffers from high variance, it means the model is overfitting — it performs very well on training data but poorly on unseen data. To handle this, several techniques can be applied. First, simplify the model by reducing its complexity or using a simpler algorithm. Regularization techniques such as L1 (Lasso), L2 (Ridge), or Elastic Net can be used to penalize large coefficients and control model flexibility. Increasing the amount of training data can also help the model generalize better. Applying cross-validation, such as k-fold, ensures the model's performance is consistent across different data subsets. Dimensionality reduction methods like PCA or feature selection can remove redundant or noisy features, further reducing overfitting. In neural networks, early stopping can be used to halt training when the validation loss starts increasing. Ensemble methods like Bagging (Random Forest) or Boosting (XGBoost with regularization) can reduce variance by combining multiple models. Finally, for image or text datasets, data augmentation techniques can artificially expand the training

data to make the model more robust. Overall, these approaches help create a more generalizable and stable model that performs well on unseen data.

42. Which Among These Is More Important Model Accuracy Or Model Performance?

43. What is active learning and where is it useful?

A subset of **machine learning** known as "active learning" allows a learning algorithm to interactively query a user to label data with the desired outputs. The algorithm actively chooses from the pool of unlabeled data the subset of examples to be labelled next in active learning. The basic idea behind the active learner algorithm concept is that if an ML algorithm could select the data it wants to learn from, it might be able to achieve a higher degree of accuracy with fewer training labels.

Active learning is particularly useful in **domains where labeling data is costly or requires expert knowledge**, such as **medical image analysis, speech recognition, fraud detection, sentiment analysis, or document classification**. For instance, in medical imaging, instead of labeling thousands of scans, radiologists can label only those images the model is uncertain about, saving time and resources while maintaining high accuracy

44. Why is Ridge Regression called Ridge?

Ridge Regression is called "*Ridge*" because of the **ridge-like shape** that appears on the **contour of the cost function** when the **L2 regularization term** is added.

In ordinary least squares (OLS) regression, the cost function is simply the sum of squared errors, which forms **elliptical contours** around the optimal solution. When Ridge Regression introduces the L2 penalty term $-(\lambda \sum \beta_i^2)$ – it effectively **adds a circular (or spherical) constraint region** on the coefficients. This constraint creates a **ridge-shaped** geometry in the cost surface, particularly along directions where coefficients can change together without greatly affecting error.

Mathematically, Ridge Regression minimizes

$$J(\beta) = \text{RSS} + \lambda \sum_{i=1}^n \beta_i^2$$

where (λ) controls the strength of regularization. The L2 term shrinks coefficients toward zero smoothly, forming a **ridge** along the path of possible solutions rather than sharp edges (as in Lasso).

In simple terms, the name "*Ridge*" comes from this **geometric interpretation** – the penalty introduces a smooth ridge in the error surface that stabilizes coefficient estimates and reduces overfitting, especially in the presence of multicollinearity.

45. State the differences between causality and correlation?

Causality and **Correlation** are two fundamental but distinct concepts in data analysis and statistics. They are often confused, but understanding their difference is critical for making correct inferences from data.

◆ Definition

- **Correlation** refers to a **statistical relationship or association** between two variables — when one changes, the other tends to change as well. However, correlation **does not imply** that one variable causes the other to change.
 - **Causality (Cause and Effect)** means that **one variable directly influences or produces a change in another**. In other words, if variable A causes variable B, changing A will lead to a predictable change in B.
-

◆ Nature of Relationship

- **Correlation**: Symmetrical — it doesn't specify direction (A can correlate with B and vice versa).
 - **Causality**: Directional — there is a clear cause (independent variable) and effect (dependent variable).
-

◆ Example

- **Correlation**: Ice cream sales and drowning incidents are correlated because both increase during summer, but one does not cause the other.
 - **Causality**: Smoking causes lung cancer — a direct, scientifically proven causal link.
-

◆ Detection

- **Correlation** is typically measured using statistical metrics like **Pearson's correlation coefficient**.

- **Causality** requires controlled experiments, **randomized trials**, or **causal inference techniques** like **Granger causality**, **do-calculus**, or **structural equation modeling**.

46. Does it make any sense to chain two different dimensionality reduction algorithms?

Yes, **it can make sense to chain two different dimensionality reduction algorithms**, depending on the dataset and the problem — and in practice, this is sometimes done to combine the **strengths of multiple techniques**.

The idea is that different dimensionality reduction methods capture **different kinds of structure** in data. For example:

- **PCA (Principal Component Analysis)** is a **linear** method that captures global variance and removes redundant features.
- **t-SNE (t-Distributed Stochastic Neighbor Embedding)** or **UMAP (Uniform Manifold Approximation and Projection)** are **nonlinear** methods that preserve **local structures** and are great for visualization.

So, a common approach is to **first apply PCA** to reduce dimensions from, say, thousands to a few hundred (removing noise and redundant features), and then apply **t-SNE or UMAP** to further reduce it to 2D or 3D for visualization or clustering.

This chaining offers several advantages:

1. **Noise Reduction:** PCA removes irrelevant variance, making the subsequent nonlinear method more effective.
2. **Computational Efficiency:** Nonlinear methods like t-SNE are computationally expensive; applying PCA first drastically reduces input size.
3. **Improved Visualization:** The combination often produces cleaner, more interpretable clusters in 2D or 3D plots.

However, there are caveats:

- If not chosen carefully, chaining can distort the underlying data structure or lead to information loss.
- It's not always necessary — for example, if your dataset is already small or linear methods are sufficient, applying multiple techniques may overcomplicate things.

✅ In summary:

Yes, chaining different dimensionality reduction methods (like PCA → t-SNE or PCA → UMAP) makes sense and is quite

common, especially when balancing **efficiency**, **noise reduction**, and **nonlinear structure preservation**. However, it should be done thoughtfully, keeping in mind the trade-off between computation and information retention.

47. Is it possible to speed up training of a bagging ensemble by distributing it across multiple servers?

Yes, absolutely. Bagging, or Bootstrap Aggregating, is inherently parallelizable because each base learner in the ensemble is trained independently on a different subset of the data. This means we can easily distribute the training process across multiple servers or processors to speed things up significantly.

For example, in a Random Forest, which is a type of bagging ensemble, each decision tree can be trained separately without depending on the others. Once all models are trained, their predictions are simply aggregated — either by averaging in regression or majority voting in classification.

Many machine learning frameworks support this kind of parallelization out of the box. For instance, in **scikit-learn**, setting `n_jobs = -1` allows the Random Forest to utilize all available CPU cores. Similarly, distributed frameworks like **Apache Spark**, **Dask**, and **H2O.ai** can spread the training process across multiple servers in a cluster for large-scale datasets.

Overall, since bagging models don't require communication between base learners during training, distributing them across multiple servers is not only possible but also a very efficient way to reduce training time and improve scalability.

48. If a Decision Tree is underfitting the training set, is it a good idea to try scaling the input features?

49. Say you trained an SVM classifier with an RBF kernel. It seems to underfit the training set: should you increase or decrease γ (gamma)? What about C?

Question: Say you trained an SVM classifier with an RBF kernel. It seems to underfit the training set: should you increase or decrease γ (gamma)? What about C?

Answer:

If your SVM with an RBF kernel is **underfitting**, you should try **increasing both γ (gamma) and C**.

Here's why:

- **γ (Gamma)** controls how far the influence of a single training example reaches.
 - A **low gamma** value means each data point has a **large radius of influence**, making the decision boundary smoother and simpler — which can lead to **underfitting**.
 - A **high gamma** value means the model focuses more on nearby points, creating a more **complex and flexible boundary** that can capture intricate patterns — but at the risk of overfitting.
 - ✓ So, if the model underfits, **increase γ** to make the decision boundary more complex.
- **C (Regularization parameter)** controls the trade-off between achieving a low training error and keeping the margin large.
 - A **small C** heavily penalizes misclassifications, enforcing a **wider margin** and simpler model — which can lead to **underfitting**.
 - A **large C** allows the model to **fit the training data more closely**, even if it means having a smaller margin — which increases flexibility.
 - ✓ So, if the model underfits, **increase C** to allow the SVM to fit the training data better.

♦ Summary:

Parameter	Effect on Model	When to Increase	When to Decrease
γ (Gamma)	Controls boundary complexity	Model underfits (too simple)	Model overfits (too wiggly)
C	Controls regularization strength	Model underfits (too constrained)	Model overfits (too flexible)

✓ In short:

If your RBF SVM is underfitting, **increase γ** to make the model more sensitive to local patterns and **increase C** to reduce regularization, allowing it to fit the training data more accurately.

50. What is cross validation and its types?

Question: What is Cross-Validation and its types?

Answer:

Cross-validation is a **model evaluation and validation technique** used to assess how well a machine learning model generalizes to unseen data. Instead of relying on a single train-test split, cross-validation involves dividing the dataset into multiple parts (or “folds”) and systematically training and testing the model on different subsets. This helps ensure that the model’s performance is **robust, reliable, and not dependent on how the data is split**.

The main idea is to use part of the data for **training** and the remaining part for **testing**, and then **repeat this process several times** to get an averaged performance metric (like accuracy, F1-score, or RMSE).

♦ Types of Cross-Validation

1. Hold-Out Validation

- The simplest form of cross-validation.
- The dataset is split into **training, validation, and test** sets (e.g., 70%-15%-15%).
- The model is trained on the training set and evaluated on the validation/test set.
- Fast but can give biased results depending on how data is split.

2. k-Fold Cross-Validation

- The dataset is divided into **k equal-sized folds**.
- The model is trained on **k-1 folds** and tested on the remaining fold.
- This process is repeated **k times**, each time using a different fold as the test set.
- The final performance is the **average of all k test results**.

- Commonly used with **k = 5 or 10**.
✔ **Advantage:** Provides a more reliable estimate of model performance than a single train-test split.
-

3. Stratified k-Fold Cross-Validation

- Similar to k-Fold, but it ensures that **each fold has the same class distribution** as the overall dataset.
 - Especially useful for **imbalanced classification problems** (e.g., 90% vs 10% class ratio).
-

4. Leave-One-Out Cross-Validation (LOOCV)

- A special case of k-Fold where **k = number of data points**.
 - Each data point acts as a test sample once, and the model is trained on all remaining points.
 - Provides an almost unbiased estimate but is **computationally expensive** for large datasets.
-

5. Time Series Cross-Validation (Rolling or Forward Chaining)

- Used for **time-dependent data** where order matters.
 - The model is trained on past data and tested on future data (e.g., train on months 1–3, test on month 4).
 - Ensures no “data leakage” from the future into the past.
-

♦ Summary Table

Type	Suitable For	Key Feature
Hold-Out	Quick checks	Simple split (train/validation/test)

k-Fold	Most models	Balanced trade-off between bias and variance
Stratified k-Fold	Imbalanced datasets	Preserves class ratio
LOOCV	Small datasets	Maximum training data, high computation
Time Series CV	Time-dependent data	Respects temporal order

✓ **In short:**

Cross-validation is a powerful technique to test a model's generalization ability. It helps prevent overfitting and ensures that the model performs consistently across different subsets of data. Among its types, **k-Fold Cross-Validation** is the most commonly used, while **Time Series CV** is ideal for sequential data.

51. How do we interpret weights in linear models?

52. Which Gradient Descent algorithm (among those we discussed) will reach the vicinity of the optimal solution the fastest? Which will actually converge?

Stochastic Gradient Descent reaches the neighborhood of the optimal solution the fastest due to frequent updates, but it doesn't fully converge. Batch Gradient Descent converges smoothly but slowly. Mini-Batch Gradient Descent offers the best trade-off — fast convergence with stability.

53. Why is it important to scale the inputs when using SVMs?

Scaling inputs is **crucial** when using Support Vector Machines (SVMs) because SVMs rely on **distance-based calculations** such as **dot products and Euclidean distances** between feature vectors to find the optimal separating hyperplane. If features are on different scales, the model can become **biased toward features with larger magnitudes**, leading to poor classification boundaries.

54. What is p value and why is it important?

55. What is OvR and OvO for multiclass classification and which machine learning algorithm supports this

56. How will you do feature selection using Lasso Regression?

57. What is the difference between loss function and cost function?

58. What are the common ways to handle missing data in a dataset?
59. What is the difference between standard scaler and minmax scaler?
What you will do if there is a categorical variable?
60. What types of model tend to overfit?
61. What are some advantages and Disadvantages of regression models and tree based models.
62. What are some important hyperparameters for XGBOOST
63. Can you tell the complete life cycle of a data science project?
64. What are the properties of a good ML model?

Property	Description
Generalization	Performs well on unseen data
Accuracy	Provides reliable predictions
Robustness	Handles noise and outliers well
Interpretability	Easy to explain and understand
Scalability	Efficient with large data
Stability	Consistent results
Fairness	No bias or discrimination
Simplicity	Minimal complexity for similar accuracy
Adaptability	Adjusts to new data or drift

65. What are the different evaluation metrices for a regression model?
66. What are the different evaluation metrices for a classification model?
67. Difference between R^2 and adjusted R^2 ? Why do you prefer adjusted r^2 ?

68. List some of the drawbacks of a Linear model

69. What do you mean by Curse of Dimensionality?

The curse of dimensionality refers to the challenges that occur when working with data having a very high number of features. As dimensionality increases, data points become sparse, distances between them lose meaning, and models struggle to identify useful patterns. This leads to increased computation time and a higher risk of overfitting. To overcome this, dimensionality reduction and feature selection techniques like PCA are commonly used.

70. What do you mean by Bias variance tradeoff?

71. Explain Kernel trick in SVM —

72. What is the main difference between Machine Learning and Data Mining?

73. Why sometimes it is needed to scale or normalise features?

74. What is the difference between Type 1 and Type 2 error ?

75. What is the difference between a Generative model vs a Discriminative model?

76. Why binary_crossentropy and categorical_crossentropy give different performances for the same problem?

77. Why does one hot encoding improve machine learning performance?

78. Considering the long list of machine learning algorithm, given a data set, how do you decide which one to use?

Choosing the right machine learning algorithm depends on several factors related to the **data, problem type**, and **constraints**. First, identify the **type of problem** — classification, regression, clustering, or recommendation. Then, consider the **size and quality of the dataset**. For example, if the dataset is small and clean, simpler models like **Logistic Regression** or **Decision Trees** might perform well. For larger and complex datasets, more powerful algorithms like **Random Forests**, **Gradient Boosting**, or **Neural Networks** could be better.

Next, check **interpretability vs. accuracy trade-off** — if you need explainable results, use linear or tree-based models; if you prioritize accuracy, deep learning models may be suitable. Also, consider **training time, computational**

resources, and whether the data is **structured or unstructured**. In practice, multiple algorithms are often tested using **cross-validation**, and the one that performs best on validation data is chosen.

79. Differentiate between wide and tall data formats?

The difference between **wide** and **tall (long)** data formats lies in how the data is structured – particularly how variables and observations are arranged.

In a **wide format**, each subject or entity has a **single row**, and repeated measurements or features are stored in **multiple columns**. This format is often used in **machine learning models** because each feature corresponds to a separate column. For example, columns like **Marks_Sem1**, **Marks_Sem2**, and **Marks_Sem3** represent multiple observations of the same variable.

In contrast, **tall (long) format** stores each observation as a **separate row**, and there is usually a column specifying the variable or measurement type. This format is common in **time series analysis** and **data visualization** (e.g., using libraries like seaborn or ggplot) because it's easier to group and aggregate data over time or categories.

- 80. What is the difference between inductive machine learning and deductive machine learning?
- 81. How will you know which machine learning algorithm to choose for your classification problem?
- 82. What is the difference between Covariance and Correlation
- 83. How will you find the correlation between a categorical variable and a continuous variable ?
- 84. What are the differences between “Bayesian” and “Frequentist” approach for Machine Learning?
- 85. What is the difference between stochastic gradient descent (SGD) and gradient descent ?
- 86. What is the difference between Gaussian Mixture Model and K-Means Algorithm?
- 87. Is more data always better?
- 88. How can you determine which features are the most important in your model?

89. Which hyper-parameter tuning strategies (in general) do you know?
90. How to select K for K-means?
91. Describe the differences between and use cases for box plots and histograms
92. How would you differentiate between Multilabel and MultiClass classification?
93. What is KL divergence, how would you define its usecase in ML?
94. Can you define the concept of Undersampling and Oversampling?
95. Considering a Long List of Machine Learning Algorithms, given a Data Set, How Do You Decide Which One to Use?
96. Explain the difference between Normalization and Standardization
97. List the most popular distribution curves along with scenarios where you will use them in an algorithm.
98. List all types of popular recommendation systems?
99. Which metrics can be used to measure correlation of categorical data?
100. Which type of sampling is better for a classification model and why?